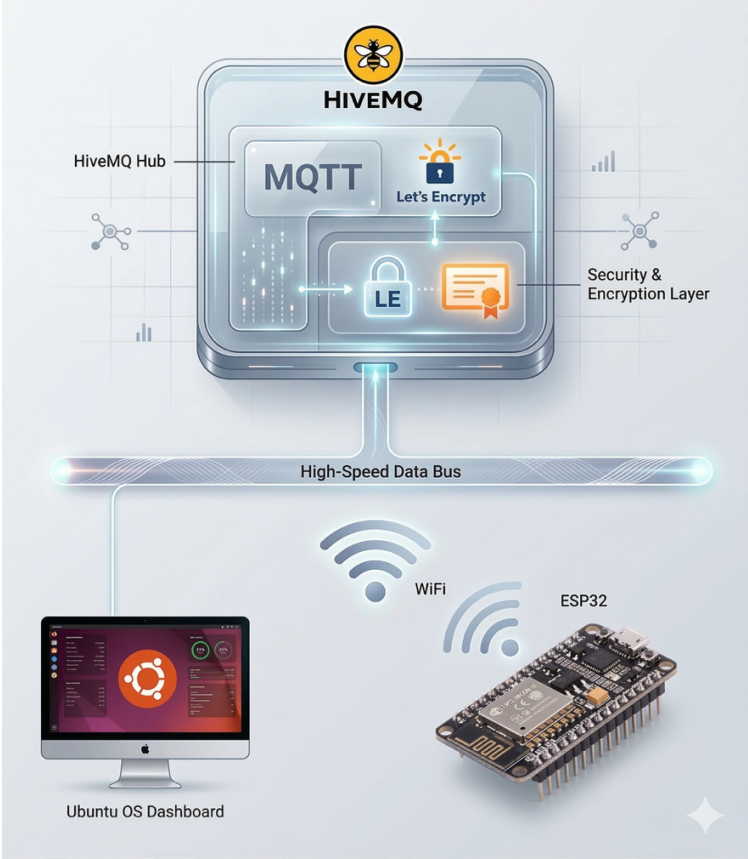
TP
MQTTS ENTRE UN ESP32 ET UN BROKER

Objectif terminal :
Sécuriser une connexion mqtt entre un ESP32 et un broker.

Objectifs intermédiaires :
- Créer un broker mqtt sur Hivemq.
- Intégrer un certificat de CA dans un code C++ ESP32

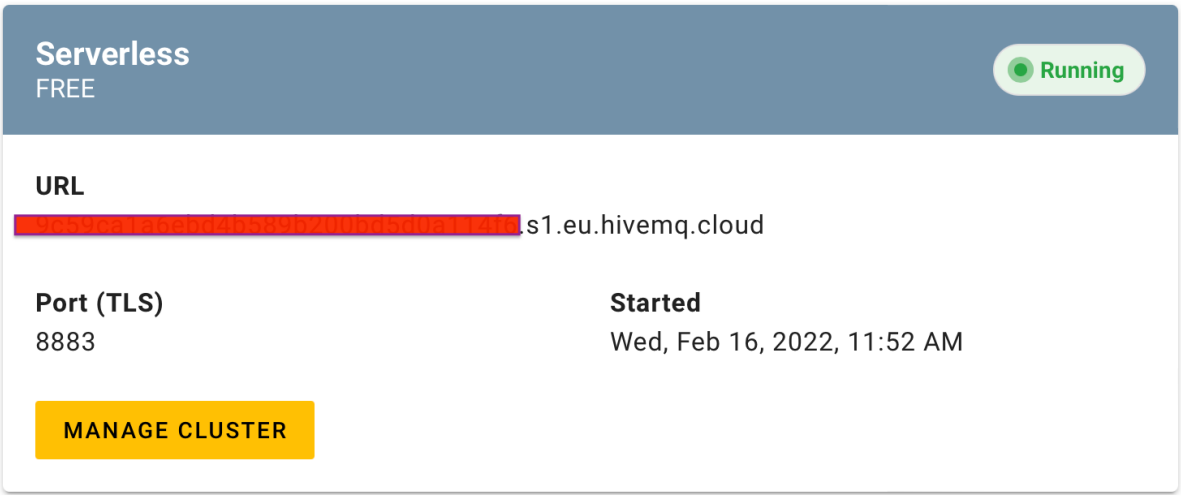
TOPOLOGIE DU TP :



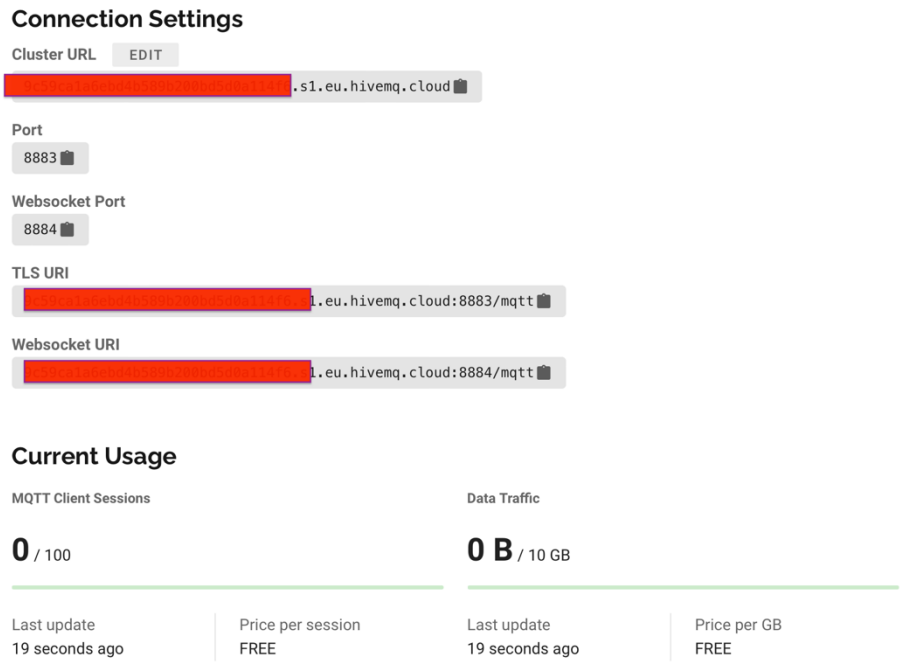
CRÉATION D'UN BROKER MQTT HIVEMQ :

1. Créer un broker sécuriser sur le cloud IOT Hivemq :

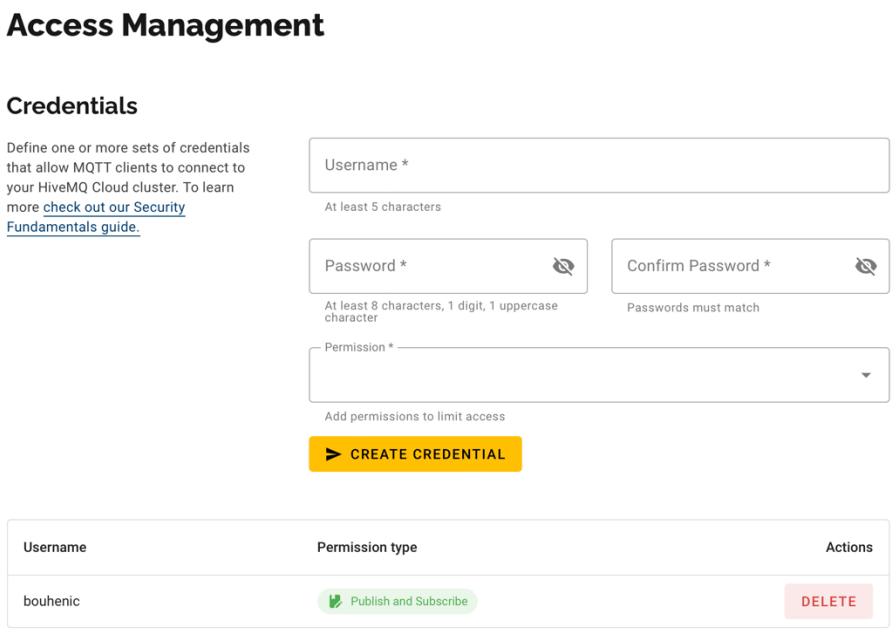
La société **HiveMQ** propose un broker dans le cloud gratuit limité à 100 sessions et 10GB par mois, ce qui est largement suffisant dans le domaine pédagogique.
Il faut créer un compte gratuit sur https://console.hivemq.cloud (identification par adresse email, compte gmail ou github)
Un fois connecté, il faut créer un **cluster** et choisir le type de serveur (AWS ou AZURE). Le choix n'entraîne aucune conséquence. Avec le compte gratuit, on peut créer 2 clusters.



Quand on clique sur **MANAGE CLUSTER** on accède à la gestion du broker.
Dans l'onglet **OVERVIEW**, on a le rappel de l'URL et des ports par lesquels on peut accéder au broker.



Dans l'onglet **ACCESS MANAGEMENT**, on peut gérer les informations d'identifications pour plusieurs utilisateurs.



Il est à noter que les mots de passe **ne peuvent être modifiés ni revisualiser ultérieurement**, il faut donc prendre la précaution de les noter dans un lieu sûr.

TEST DU BROKER HIVEMQ:

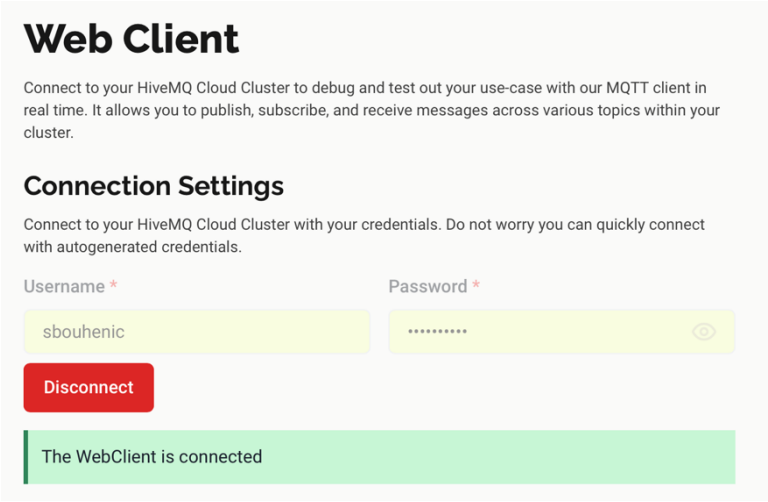
1. Depuis le terminal, saisir le commande mosquitto suivante permettant l'abonnement au topic « topic/test »:

```
mosquitto_sub -h 5exxxxxxxxxxxxxxxxxxd80fd.s1.eu.hivemq.cloud -p
8883 -u xxxxxxxx -P xxxxxxxxxxx -t topic/test
```

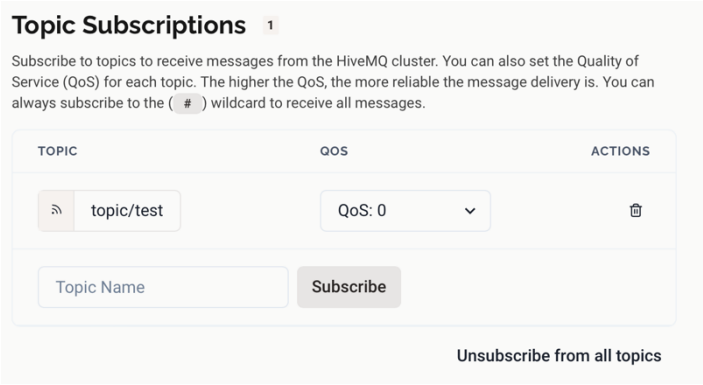
Hivemq propose un client mqtt. Nous allons l'utiliser pour publier un message sur le topic « topic/test ».

2. Se connecter sur Hivemq et sélectionner l'onglet WEB CLIENT :

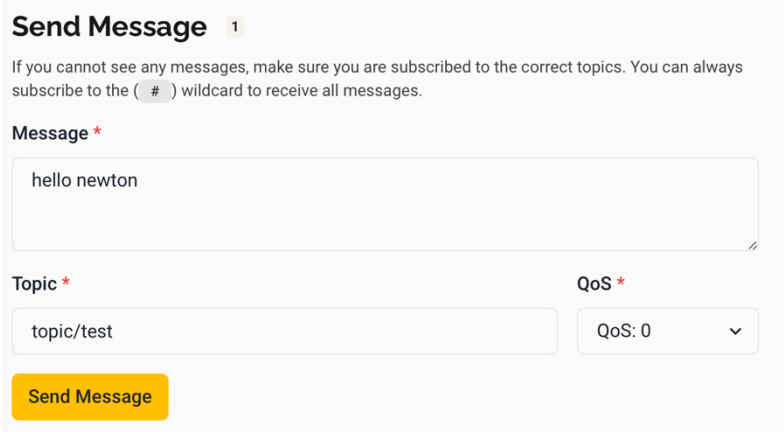
3. Saisir vos identifiants/mot de passe de façon à connecter le client sur le broker.



On peut aussi s'abonner au topic depuis la fenêtre suivante :



4. Publier un message à l'aide de la fenêtre suivante :



Vous testez sur le terminal :

```
Hello newton
```

Nous n'avons pas eu besoin de spécifier le certificat du CA dans la commande mosquitto avec l'option –cafile. En effet, hivemq utilise un certificat connu let's encrypt. On peut le vérifier en saisissant la commande openssl suivante :

```
openssl s_client -connect 5e7xxxxxxxxxxxxx.s1.eu.hivemq.cloud:8883
-servername 5e7xxxxxxxxxxxxx.s1.eu.hivemq.cloud -showcerts
```

Il y a beaucoup de chose dans la réponse. On voit les certificats du serveur et du CA intermédiaire. On voit également la chaine de certification qui indique la profondeur (depth).

```
depth=2 C = US, O = Internet Security Research Group, CN = ISRG Root X1
verify return:1
depth=1 C = US, O = Let's Encrypt, CN = R11
verify return:1
depth=0 CN = *.s1.eu.hivemq.cloud
verify return:1
```

- **depth=0 → Le certificat du serveur** (*s1.eu.hivemq.cloud*).
- **depth=1 → Le certificat intermédiaire** (*Let's Encrypt R11*).
- **depth=2 → Le certificat racine** (*ISRG Root X1*).

👉 Le fait que *"Let's Encrypt R11"* soit à depth=1 et que son **issuer** soit ISRG Root X1 indique clairement qu'il s'agit **d'un certificat intermédiaire**.

Le certificat racine n'est pas transmis, il est présent dans la banque de certificat. On peut la visualiser en saisissant :

```
ls /etc/ssl/certs/
```

PROGRAMMATION DE L'ESP32 :

1. Lancer l'IDE Arduino en saisissant sur le terminal.

2. Lancer une deuxième fenêtre de terminal.

3. Télécharger le fichier arduino mqtts.ino en saisissant :

```
git clone https://github.com/bouhenic/FormationIOTSecu/mqtts
cd FormationIOTSecu/mqtts
```

4. Depuis Arduino, Ouvrir le fichier mqtts.ino

```
#include <WiFi.h>
#include <WiFiClientSecure.h>
#include <PubSubClient.h>

const char* ssid = "xxxxxxxxxx"; //Compléter ici
const char* password = "xxxxxxxxxx"; //Compléter ici
const char* mqtt_server = "5exxxxxxxxxxxxxxxxxxxxx75efd80fd.s1.eu.hivemq.cloud";
const int mqtt_port = 8883;
const char* mqtt_user = "xxxxxxxxxx"; //Compléter ici
const char* mqtt_pass = "xxxxxxxxxx"; //Compléter ici


// Certificat racine ISRG Root X1 (Let's Encrypt)
const char* ca_cert = \
"-----BEGIN CERTIFICATE-----\n" \
"MIIFazCCA1OgAwIBAgIRAIIQz7DSQONZRGPgu2OCiwAwDQYJKoZIhvcNAQELBQAw\n" \
"TzELMAkGA1UEBhMCVVMxKTAnBgNVBAoTIEludGVybmV0IFNlY3VyaXR5IFJlc2Vh\n" \
"cmNoIEdyb3VwMRUwEwYDVQQDEwxJU1JHIFJvb3QgWDEwHhcNMTUwNjA0MTEwNDM4\n" \
"WhcNMzUwNjA0MTEwNDM4WjBPMQswCQYDVQQGEwJVUzEpMCcGA1UEChMgSW50ZXJu\n" \
"ZXQgU2VjdXJpdHkgUmVzZWFyY2ggR3JvdXAxFTATBgNVBAMTDElTUkcgUm9vdCBY\n" \
"MTCCAiIwDQYJKoZIhvcNAQEBBQADggIPADCCAgoCggIBAK3oJHP0FDfzm54rVygc\n" \
"h77ct984kIxuPOZXoHj3dcKi/vVqbvYATyjb3miGbESTtrFj/RQSa78f0uoxmyF+\n" \
"0TM8ukj13Xnfs7j/EvEhmkvBioZxaUpmZmyPfjxwv60pIgbz5MDmgK7iS4+3mX6U\n" \
"A5/TR5d8mUgjU+g4rk8Kb4Mu0UlXjIB0ttov0DiNewNwIRt18jA8+o+u3dpjq+sW\n" \
"T8KOEUt+zwvo/7V3LvSye0rgTBIlDHCNAymg4VMk7BPZ7hm/ELNKjD+Jo2FR3qyH\n" \
"B5T0Y3HsLuJvW5iB4YlcNHlsdu87kGJ55tukmi8mxdAQ4Q7e2RCOFvu396j3x+UC\n" \
"B5iPNgiV5+I3lg02dZ77DnKxHZu8A/lJBdiB3QW0KtZB6awBdpUKD9jf1b0SHzUv\n" \
"KBds0pjBqAlkd25HN7rOrFleaJ1/ctaJxQZBKT5ZPt0m9STJEadao0xAH0ahmbWn\n" \
"OlFuhjuefXKnEgV4We0+UXgVCwOPjdAvBbI+e0ocS3MFEvzG6uBQE3xDk3SzynTn\n" \
"jh8BCNAw1FtxNrQHusEwMFxIt4I7mKZ9YIqioymCzLq9gwQbooMDQaHWBfEbwrbw\n" \
"qHyGO0aoSCqI3Haadr8faqU9GY/rOPNk3sgrDQoo//fb4hVC1CLQJ13hef4Y53CI\n" \
"rU7m2Ys6xt0nUW7/vGT1M0NPAgMBAAGjQjBAMA4GA1UdDwEB/wQEAwIBBjAPBgNV\n" \
"HRMBAf8EBTADAQH/MB0GA1UdDgQWBBR5tFnme7bl5AFzgAiIyBpY9umbbjANBgkq\n" \
"hkiG9w0BAQsFAAOCAgEAVR9YqbyyqFDQDLHYGmkgJykIrGF1XIpu+ILlaS/V9lZL\n" \
"ubhzEFnTIZd+50xx+7LSYK05qAvqFyFWhfFQDlnrzuBZ6brJFe+GnY+EgPbk6ZGQ\n" \
"3BebYhtF8GaV0nxvwuo77x/Py9auJ/GpsMiu/X1+mvoiBOv/2X/qkSsisRcOj/KK\n" \
"NFtY2PwByVS5uCbMiogziUwthDyC3+6WVwW6LLv3xLfHTjuCvjHIInNzktHCgKQ5\n" \
"ORAzI4JMPJ+GslWYHb4phowim57iaztXOoJwTdwJx4nLCgdNbOhdjsnvzqvHu7Ur\n" \
"TkXWStAmzOVyyghqpZXjFaH3pO3JLF+l+/+sKAIuvtd7u+Nxe5AW0wdeRlN8NwdC\n" \
"jNPElpzVmbUq4JUagEiuTDkHzsxHpFKVK7q4+63SM1N95R1NbdWhscdCb+ZAJzVc\n" \
"oyi3B43njTOQ5yOf+1CceWxG1bQVs5ZufpsMljq4Ui0/1lvh+wjChP4kqKOJ2qxq\n" \
"4RgqsahDYVvTH9w7jXbyLeiNdd8XM2w9U/t7y0Ff/9yi0GE44Za4rF2LN9d11TPA\n" \
"mRGunUHBcnWEvgJBQl9nJEiU0Zsnvgc/ubhPgXRR4Xq37Z0j4r7g1SgEEzwxA57d\n" \
"emyPxgcYxn/eR44/KJ4EBs+lVDR3veyJm+kXQ99b21/+jh5Xos1AnX5iItreGCc=\n" \
"-----END CERTIFICATE-----\n";
```

```
WiFiClientSecure wifiClient;
PubSubClient client(wifiClient);

void connectToWiFi() {
  Serial.println("Connexion au WiFi...");
  WiFi.begin(ssid, password);
  while (WiFi.status() != WL_CONNECTED) {
    delay(500);
    Serial.print(".");
  }
  Serial.println("\nConnecté au WiFi");
}

void connectToMQTT() {
  while (!client.connected()) {
    Serial.println("Connexion au broker MQTT...");
    if (client.connect("ESP32Client", mqtt_user, mqtt_pass)) {
      Serial.println("Connecté au broker MQTT");
      client.subscribe("xxxxx/xxxxxx"); //Compléter ici le topic
    } else {
      Serial.print("Échec de connexion, rc=");
      Serial.println(client.state());
      delay(5000);
    }
  }
}

void callback(char* topic, byte* payload, unsigned int length) {
  Serial.print("Message reçu sur le sujet : ");
  Serial.println(topic);

  Serial.print("Message : ");
  for (unsigned int i = 0; i < length; i++) {
    Serial.print((char)payload[i]);
  }
  Serial.println();
}

void setup() {
  Serial.begin(115200);

  connectToWiFi();

  wifiClient.setCACert(ca_cert); // Configure le certificat pour la sécurité
  client.setServer(mqtt_server, mqtt_port);
  client.setCallback(callback);

  connectToMQTT();
}

void loop() {
  if (!client.connected()) {
    connectToMQTT();
  }
  client.loop();
}
```

5. Modifier les paramètres suivants du programme fourni (ssid, password, mqtt_server, mqtt_user, mqtt_pass et topic).

6. Sélectionner la carte « ESP32 Dev Module » dans l'onglet Tools/Boards Manager

7. Téléverser le programme dans l'ESP32.

TEST DU PROGRAMME TÉLÉVERSÉ :

5. Saisir la commande mosquitto suivante pour publier un message en remplaçant l'url de votre broker, le topic, le message, votre username et votre password :

```
Mosquitto_pub -h 5e7xxxxxxxxxxx.s1.eu.hivemq.cloud -p 8883 -u
xxxxxx -P xxxxxx -t formation/IOTsecurity -m "Hello IOT security"
```

6. Sur le serial monitor d'Arduino, vérifier la réception du message :

ANNEXE : Explication détaillée

1. **Inclusion des bibliothèques**

```
#include <WiFi.h>
#include <WiFiClientSecure.h>
#include <PubSubClient.h>
```

2. **Déclaration des informations de connexion**

```
const char* ssid = "xxxxxxxxxx"; //Compléter ici
const char* password = "xxxxxxxxxx"; //Compléter ici
const char* mqtt_server = "5exxxxxxxxxxxxxxxxxxxxx75efd80fd.s1.eu.hivemq.cloud";
const int mqtt_port = 8883;
const char* mqtt_user = "xxxxxxxxxx"; //Compléter ici
const char* mqtt_pass = "xxxxxxxxxx"; //Compléter ici
```

- **ssid et password** : Informations d'identification du réseau Wi-Fi.
- **mqtt_server** : Adresse du **broker MQTT**, ici hébergé chez **HiveMQ Cloud**.
- **mqtt_port = 8883** : Port standard pour MQTT **sécurisé (MQTTs)**.
- **mqtt_user et mqtt_pass** : Identifiants pour s'authentifier auprès du broker.

3. **Certificat racine Let's Encrypt**

```
const char* ca_cert = \
"-----BEGIN CERTIFICATE-----\n" \
"MIIFazCCA1OgAwIBAgIRAIIQz7DSQONZRGPgu2OCiwAwDQYJKoZIhvcNAQELBQAw\n" \
"... (contenu du certificat) ...\n" \
"-----END CERTIFICATE-----\n";
```

- Ce certificat **ISRG Root X1** provient de **Let's Encrypt**.
- Il permet à l'ESP32 de **vérifier l'identité du serveur MQTT** pour éviter les attaques **MITM (Man-In-The-Middle)**.

4. **Création des objets WiFi et MQTT**

```
WiFiClientSecure wifiClient;
PubSubClient client(wifiClient);
```

- **WiFiClientSecure wifiClient;**
  - Initialise un client Wi-Fi sécurisé **(SSL/TLS)** pour la connexion MQTT.
- **PubSubClient client(wifiClient);**
  - Crée un client MQTT sécurisé en utilisant wifiClient.

5. **Fonction de connexion au Wi-Fi**

```
void connectToWiFi() {
  Serial.println("Connexion au WiFi...");
  WiFi.begin(ssid, password);
  while (WiFi.status() != WL_CONNECTED) {
    delay(500);
    Serial.print(".");
  }
  Serial.println("\nConnecté au WiFi");
}
```

- **WiFi.begin(ssid, password);** → Démarre la connexion au réseau Wi-Fi.
- **Boucle while (WiFi.status() != WL_CONNECTED)**
  - Attend que l'ESP32 soit connecté.
- Affiche des "." toutes les 500 ms pour montrer l'attente.

6. Connexion au broker MQTT

```
void connectToMQTT() {
  while (!client.connected()) {
    Serial.println("Connexion au broker MQTT...");
    if (client.connect("ESP32Client", mqtt_user, mqtt_pass)) {
      Serial.println("Connecté au broker MQTT");
      client.subscribe("xxxxx/xxxxxx"); // Compléter ici le topic
    } else {
      Serial.print("Échec de connexion, rc=");
      Serial.println(client.state());
      delay(5000);
    }
  }
}
```

- **client.connected()**
  - Vérifie si l'ESP32 est déjà connecté au broker MQTT.
- **Tentative de connexion avec client.connect("ESP32Client", mqtt_user, mqtt_pass)**
  - ESP32Client : Nom de l'ESP32 pour le broker MQTT.
  - **mqtt_user et mqtt_pass** : Identifiants pour l'authentification.
- **Si la connexion réussit** :
  - Affiche "Connecté au broker MQTT".
  - Souscrit au **topic** "xxxxx/xxxxxx" (doit être remplacé par un vrai topic).
- **Si la connexion échoue** :
  - Affiche l'erreur MQTT (client.state()).
  - Attends **5 secondes** avant de retenter.

7. Fonction de réception des messages MQTT

```
void callback(char* topic, byte* payload, unsigned int length) {
  Serial.print("Message reçu sur le sujet : ");
  Serial.println(topic);

  Serial.print("Message : ");
  for (unsigned int i = 0; i < length; i++) {
    Serial.print((char)payload[i]);
  }
  Serial.println();
}
```

- Cette fonction est appelée **automatiquement** lorsqu'un message MQTT est reçu.
- **Affiche le topic** et **le message** reçu en console série.

8. Initialisation dans setup()

```
void setup() {
  Serial.begin(115200);

  connectToWiFi();

  wifiClient.setCACert(ca_cert); // Configure le certificat pour la sécurité
  client.setServer(mqtt_server, mqtt_port);
  client.setCallback(callback);

  connectToMQTT();
}
```

- **Serial.begin(115200);** → Initialise la console série pour le débogage.
- **connectToWiFi();** → Connecte l'ESP32 au Wi-Fi.
- **wifiClient.setCACert(ca_cert);**
  - Configure le **certificat racine Let's Encrypt** pour la connexion sécurisée.

- **client.setServer(mqtt_server, mqtt_port);**
    - Définit l'adresse et le port du **broker MQTT**.
- **client.setCallback(callback);**
    - Associe la fonction callback() pour recevoir les messages MQTT.
- **connectToMQTT();** → Connecte l'ESP32 au **broker MQTT sécurisé**.

9. **Boucle principale loop()**

```
void loop() {
  if (!client.connected()) {
    connectToMQTT();
  }
  client.loop();
}
```

- **Si la connexion MQTT est perdue** → La fonction connectToMQTT(); tente de la rétablir.
- **client.loop();** → Maintient la connexion MQTT active et gère la réception des messages.